

数据与算法 - 第8讲

图（二）、查找

冯伟 (fengwei@tsinghua.edu.cn)

2024年11月5日





提纲

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法

最短路径问题

带权有向图中每条路径都可以定义路径权，其值为该路径上各条边的权值之和；从顶点 v (源点) 到顶点 w (汇点) 的最短路径是路径权最小的简单路径（最短路径可能不唯一）



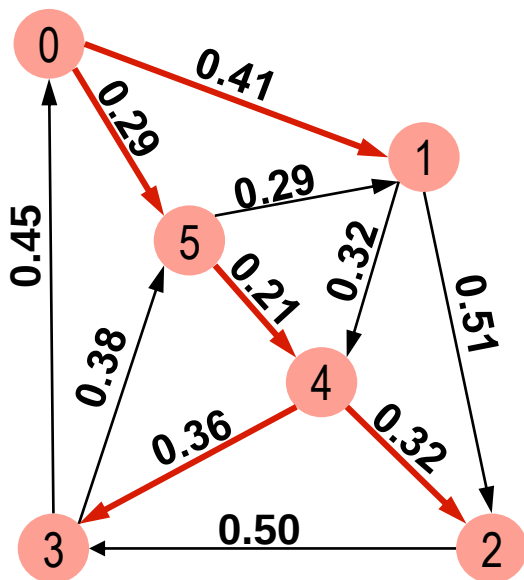


最短路径问题

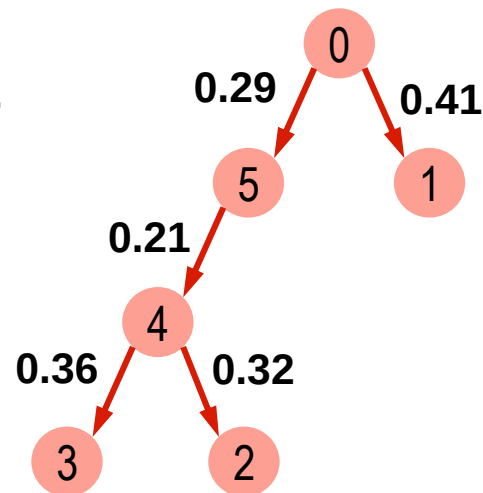
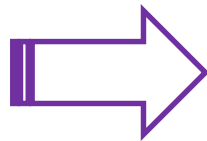
- 源点-汇点最短路径 (Single-pair Shortest Path)
- 单源最短路径 (Single-source Shortest Path): 给定一个起始顶点, 找出从该源点到图中其他顶点的最短路径
- 单汇最短路径 (Single-destination Shortest Path): 给定一个汇点, 找出从图中其他顶点到该汇点的最短路径
- 全源最短路径 (All-pairs Shortest Path): 找出图中任意一个有序顶点对的最短路径

单源最短路径问题

某最短路径经过 v_1 和 v_2 ，则 v_1 和 v_2 之间的路径也是他们之间的最短路径



构建最短路径树



- 给定一个起始顶点 s ，找出从 s 到图中其他顶点的最短路径
- 可以看成构建了一棵最短路径树 (SPT) (n 个顶点, $n-1$ 条边)
- 根结点为起始点 s ，由原图中从顶点 s 到其余各顶点的最短路径构成
- 从一个顶点出发可能有多个SPT



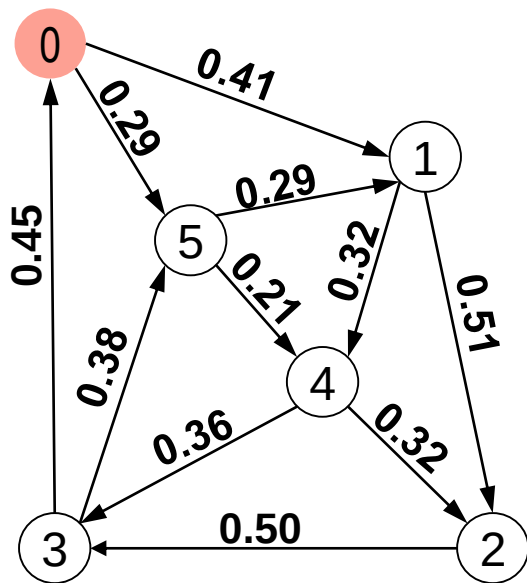
单源最短路径 - Dijkstra算法

构造带权有向图的最短路径树（SPT）实现单源最短路径算法

1. 初始化SPT顶点集 $S = \{s\}$, $s \in S$ 为源点, V 为图的顶点集, $V - S$ 表示图中所有不属于SPT的顶点
2. 沿源点 s 发出的边进行扩展, 更新源点 s 到 $V - S$ 中各顶点的最短距离
3. 找出 $V - S$ 中距源点 s 最近的顶点 v , 将顶点 v 和相应的边放入SPT
4. 沿顶点 v 发出的边更新源点到 $V - S$ 中各顶点的距离
5. 不断重复步骤3和4, S 集合不断扩大, 当 $S = V$, 求解完毕



单源最短路径 - Dijkstra算法

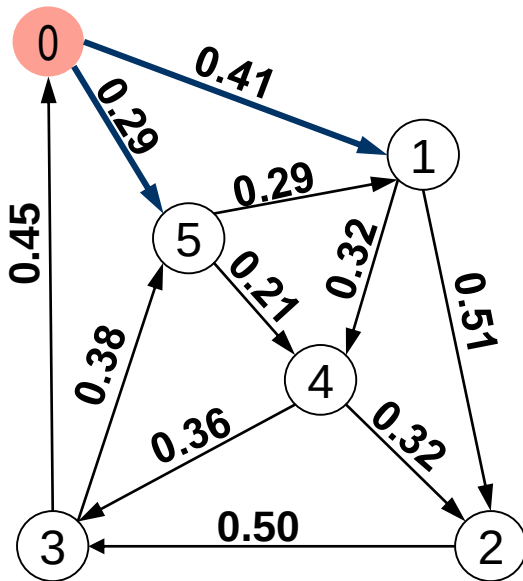


SPT {0}

0	0	0
1	-1	
2	-1	
3	-1	
4	-1	
5	-1	



单源最短路径 - Dijkstra算法



0	0	0
1	0.41	0-1
2	-1	
3	-1	
4	-1	
5	0.29	0-5

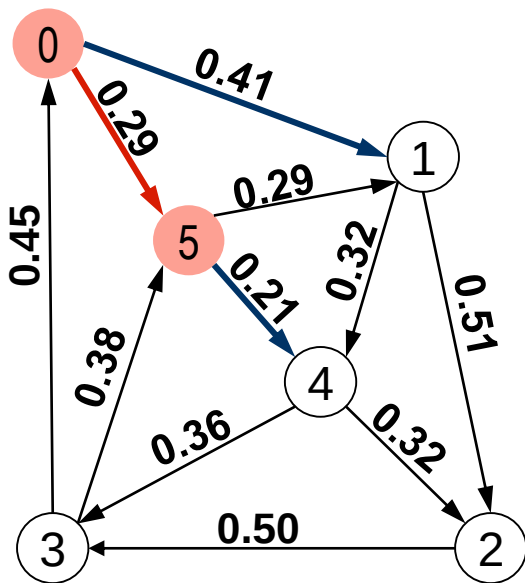
SPT {0}

Edge 0-5 0.29

0-1 0.41



单源最短路径 - Dijkstra算法



0	0	0
1	0.41	0-1
2	-1	
3	-1	
4	0.50	0-5-4
5	0.29	0-5

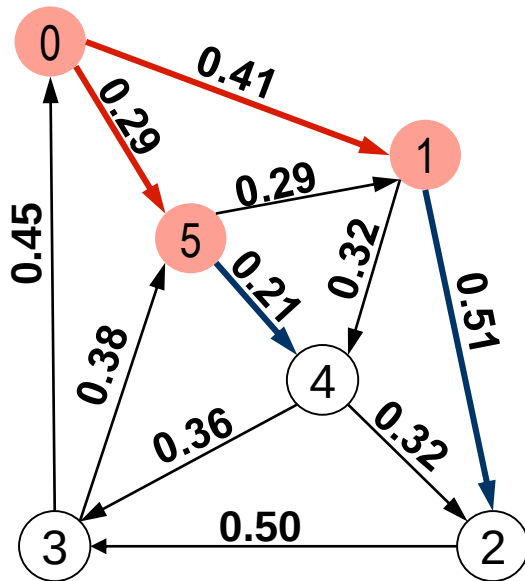
SPT {0, 5}

Edge 0-1 0.41

0-5-4 0.50



单源最短路径 - Dijkstra算法



SPT {0, 5, 1}

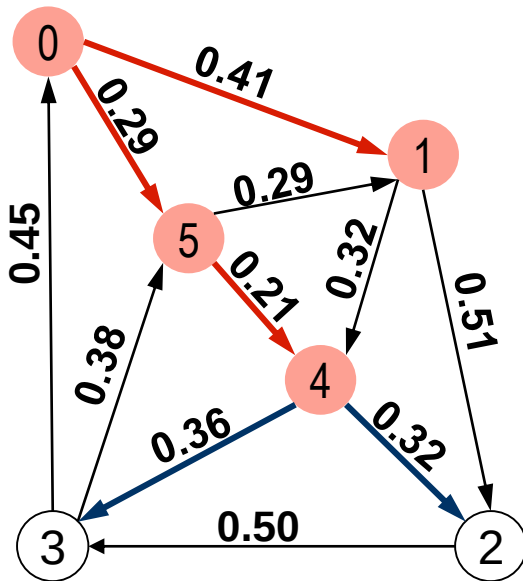
Edge 0-5-4 0.5

0-1-2 0.92

0	0	0
1	0.41	0-1
2	0.92	0-1-2
3	-1	
4	0.50	0-5-4
5	0.29	0-5



单源最短路径 - Dijkstra算法



0	0	0
1	0.41	0-1
2	0.82	0-5-4-2
3	0.86	0-5-4-3
4	0.50	0-5-4
5	0.29	0-5

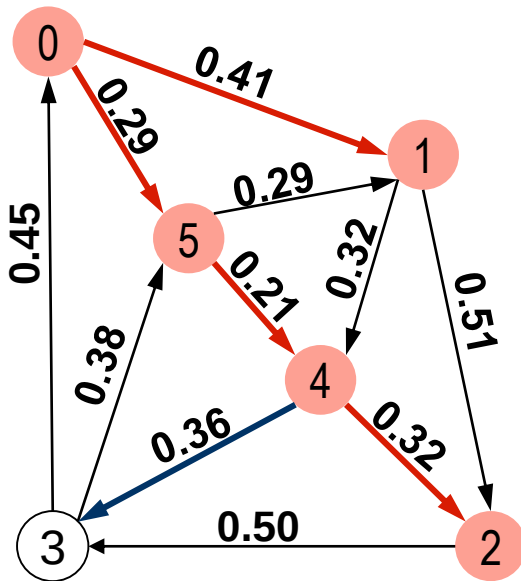
SPT {0, 5, 1, 4}

Edge 0-5-4-2 0.82

0-5-4-3 0.86



单源最短路径 - Dijkstra算法

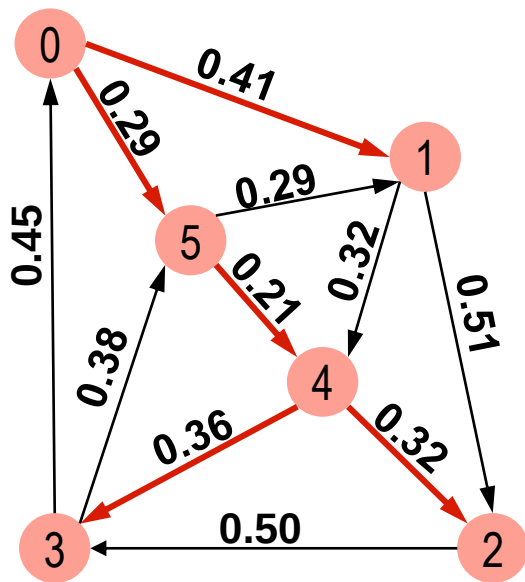


SPT {0, 5, 1, 4, 2}
Edge 0-5-4-3 0.86

0	0	0
1	0.41	0-1
2	0.82	0-5-4-2
3	0.86	0-5-4-3
4	0.50	0-5-4
5	0.29	0-5



单源最短路径 - Dijkstra算法



SPT {0, 5, 1, 4, 2, 3}

0	0	0
1	0.41	0-1
2	0.82	0-5-4-2
3	0.86	0-5-4-3
4	0.50	0-5-4
5	0.29	0-5



单源最短路径 - Dijkstra算法

➤ Dijkstra算法与Prim算法类似

□ Prim算法取非MST中距离MST最近的顶点及其对应的边

□ Dijkstra算法取从源点到非SPT顶点的最短路径的最后一条边

➤ Dijkstra算法的时间复杂度为 $O(n^2)$ ，适用于稠密图

➤ Dijkstra算法属于贪心算法（每次只考虑最短的路径）

Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps*

Bernhard Haeupler
INSAIT, Sofia University
“St. Kliment Ohridski”
& ETH Zurich

Richard Hladík
INSAIT, Sofia University
“St. Kliment Ohridski”
& ETH Zurich

Václav Rozhoň
INSAIT, Sofia University
“St. Kliment Ohridski”

Robert E. Tarjan
Princeton University

Jakub Tětek
INSAIT, Sofia University
“St. Kliment Ohridski”

**FOCS 2024: Dijkstra算
法被证明普遍最优：最坏
情况性能也最优！**



全源最短路径问题

- 求出图中任意两个顶点间的最短路径
- 可以对每个顶点应用Dijkstra算法，时间复杂度为 $O(n^3)$
- 另一种算法：弗洛伊德(Floyd)算法，时间复杂度也为 $O(n^3)$



全源最短路径 - Floyd算法

- 初始化 $n \times n$ 的矩阵 D_0 （即邻接矩阵），为图的边值矩阵
- 按照下面公式进行迭代

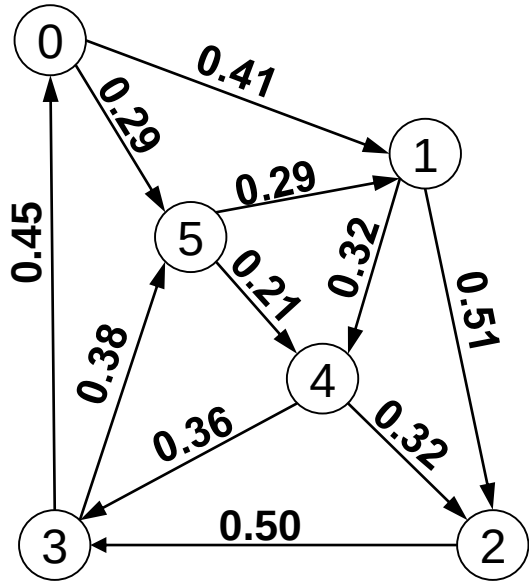
$$D_k[i][j] = \min\{D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j]\}$$

$$k = 1, \dots, n - 1$$

- $D_k[i][j]$ 即为从顶点 i 到 j 的最短路径长度，这条路径不经过编号大于 k 的顶点
- 经过 n 次迭代，最后得到全源最短路径矩阵



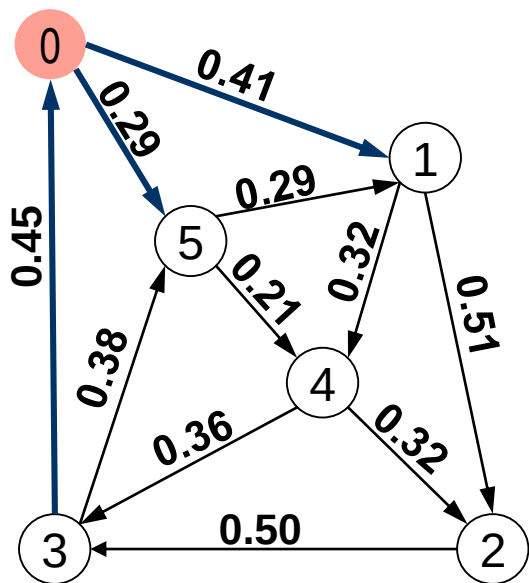
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	∞	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



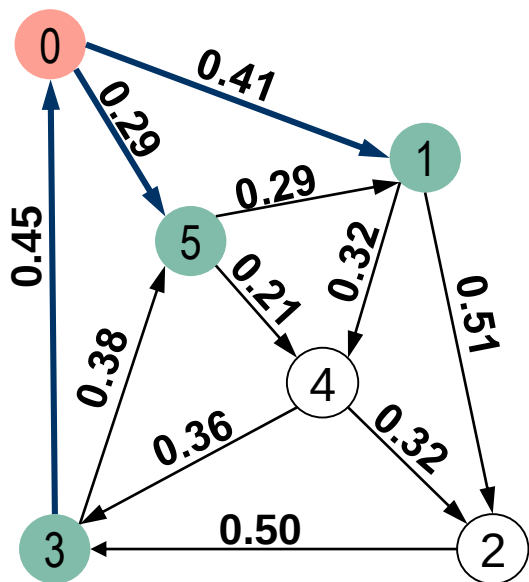
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	∞	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



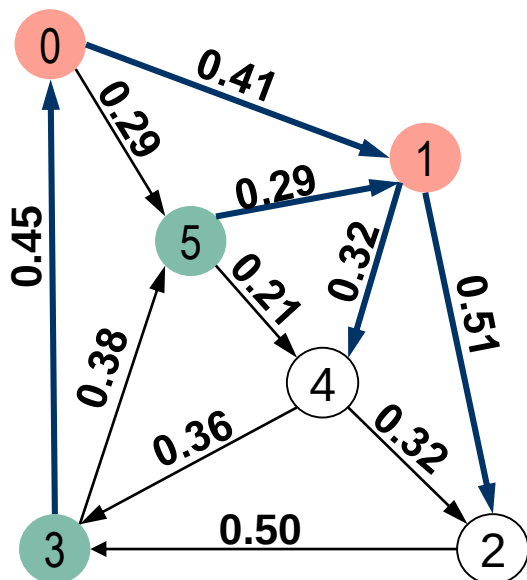
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



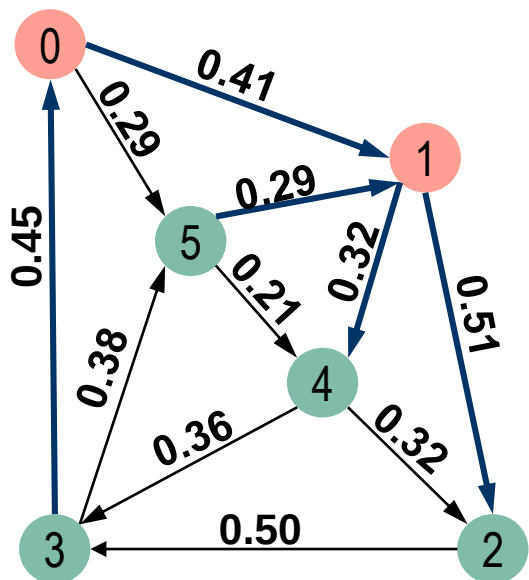
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



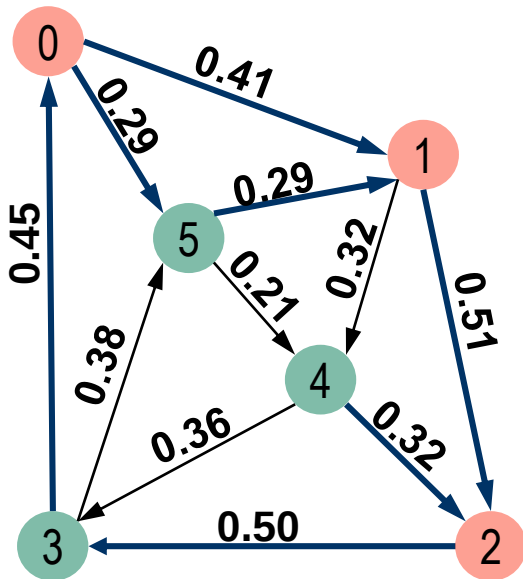
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	∞	0.73	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	∞	0.21	0



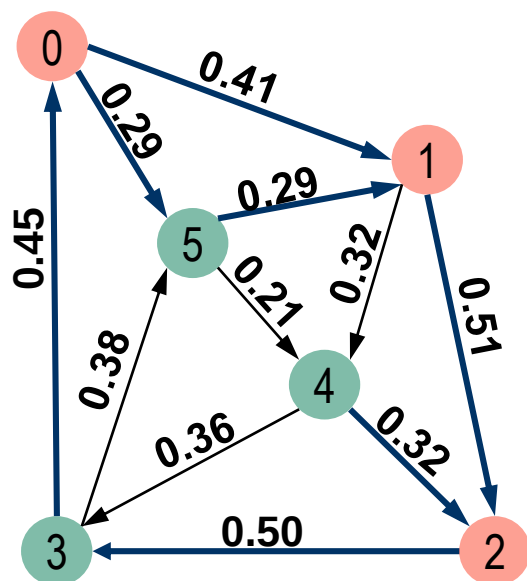
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	∞	0.73	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	∞	0.21	0



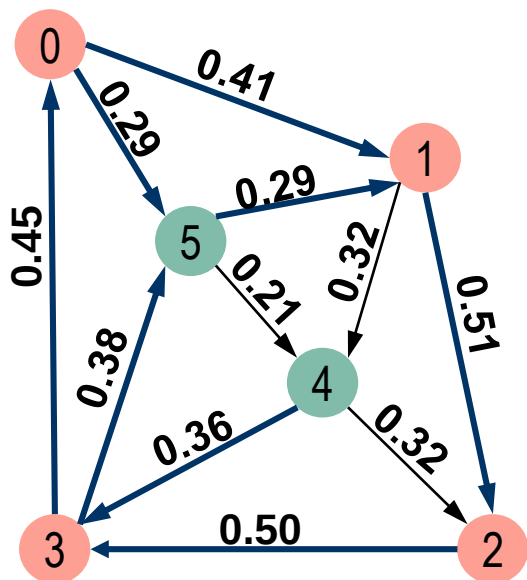
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	∞	0	0.51	1.01	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	1.30	0.21	0



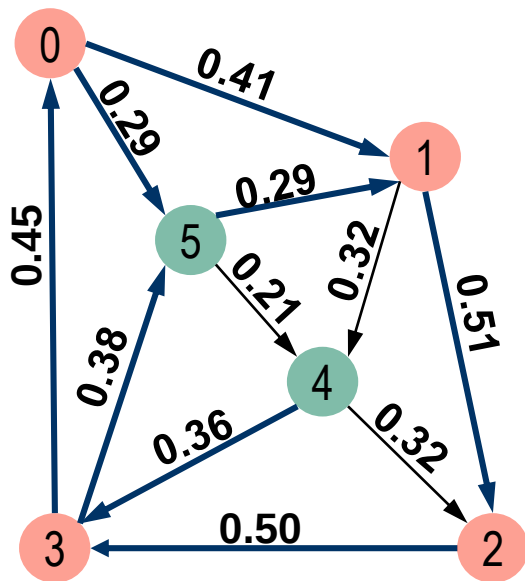
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	∞	0	0.51	1.01	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	1.30	0.21	0



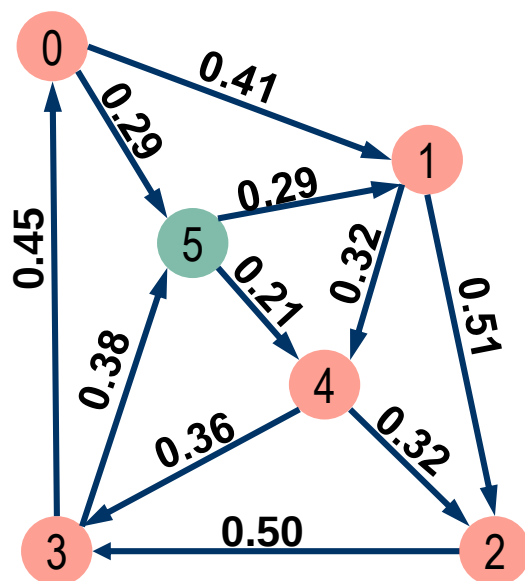
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	1.46	0	0.51	1.01	0.32	1.39
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.75	0.29	0.80	1.30	0.21	0



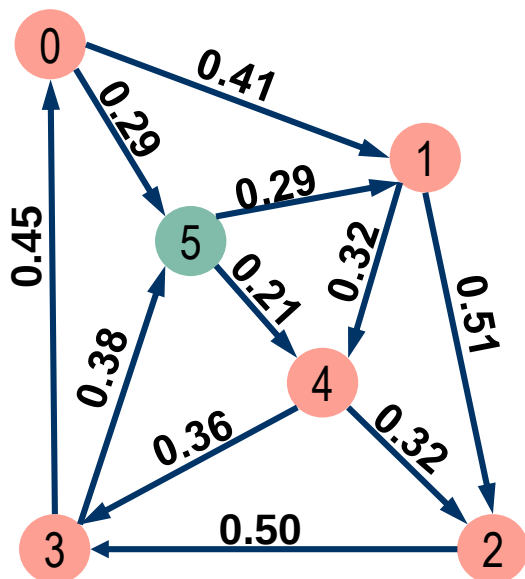
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	1.46	0	0.51	1.01	0.32	1.39
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.75	0.29	0.80	1.30	0.21	0



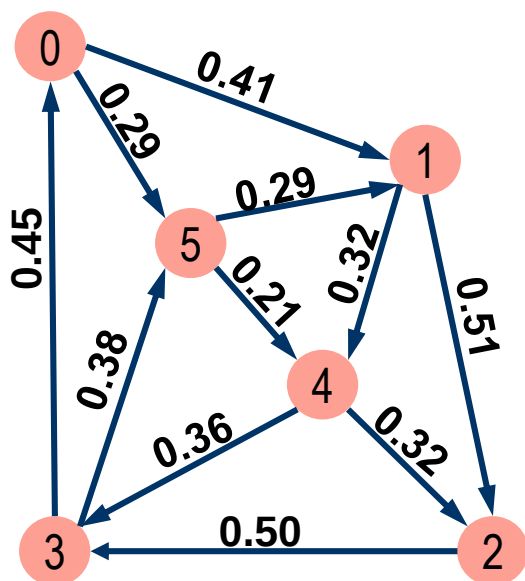
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.09	0.73	0.29
1	1.13	0	0.51	0.68	0.32	1.06
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0



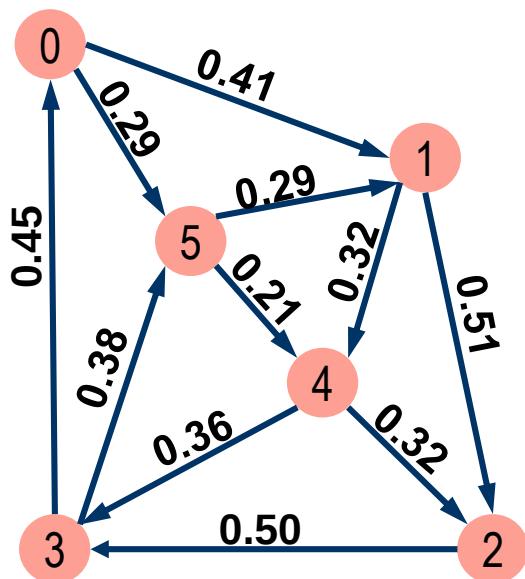
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.09	0.73	0.29
1	1.13	0	0.51	0.68	0.32	1.06
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0



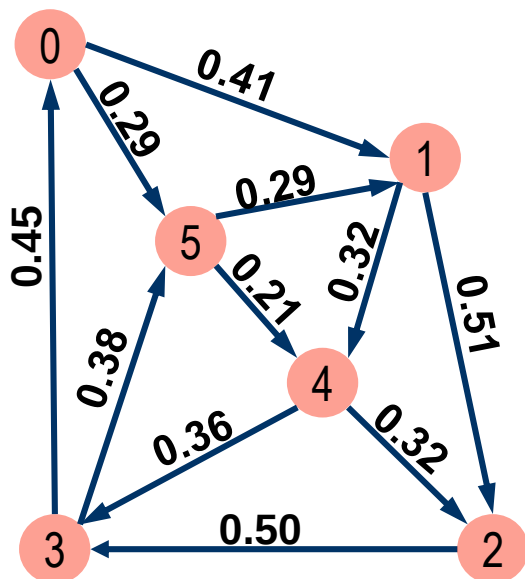
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.82	0.86	0.50	0.29
1	1.13	0	0.51	0.68	0.32	1.06
2	0.95	1.17	0	0.50	1.09	0.88
3	0.45	0.67	0.91	0	0.69	0.38
4	0.81	1.03	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0



全源最短路径 - Floyd算法

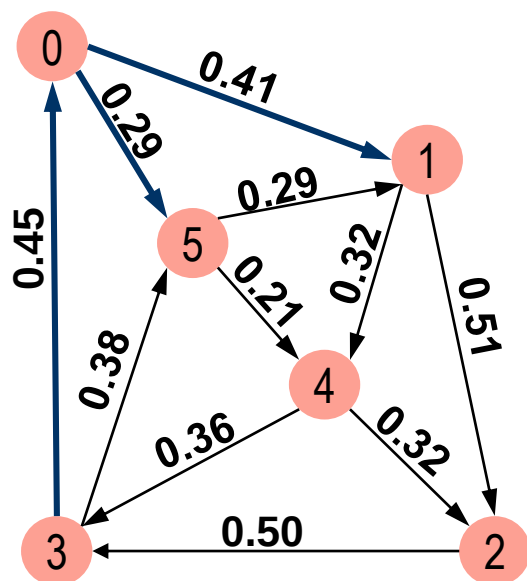


	0	1	2	3	4	5
0	0	0.41	0.82	0.86	0.50	0.29
1	1.13	0	0.51	0.68	0.32	1.06
2	0.95	1.17	0	0.50	1.09	0.88
3	0.45	0.67	0.91	0	0.69	0.38
4	0.81	1.03	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0

边值矩阵



全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	01	0542	0543	054	05
1	1430	0	12	143	14	1435
2	230	2351	0	23	2354	235
3	30	351	3542	0	354	35
4	430	4351	42	43	0	435
5	5430	51	542	543	54	0

路径矩阵

提纲

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法





查找（搜索，Search）

- 查找是符号处理中的常见问题，应用广泛
 - 信息检索，搜索引擎
 - 查表法：三角函数，概率分布
- 关键字（当前关心的数据项）与数据元素（由若干数据项组成）
- 数据结构→查找 vs. 查找需求→数据结构
- 静态查找结构、动态查找结构（有插入、删除操作）
- 度量查找算法的性能：时间和空间复杂度
 - 基本操作是关键字比较，比较次数决定了算法的时间复杂度



查找（搜索，Search）

平均查找长度(Average search length, ASL)作为衡量算法时间效率的准则

- 在查找过程中，为确定目标数据元素的位置，需要进行关键字比较次数的期望值

$$ASL = \sum_{i=0}^{n-1} P_i C_i$$

- P_i : 第 i 个元素被查找的概率
- C_i : 第 i 个元素被查找所需要的比较次数
- ASL 越小，查找的性能越好
- 如果考虑查找失败的情况（算法鲁棒性考虑），需对上式做修正

提纲

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法





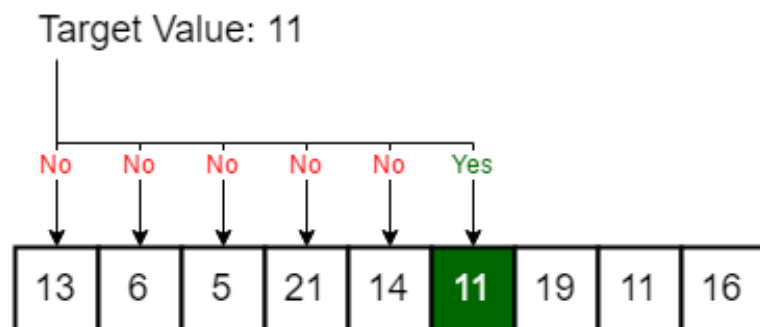
线性查找表

- 用于查找的最简单的数据结构（线性结构）
- 线性查找表：数据元素依次连续存储在一个线性表中
- 按关键字(Key)查找
 - 1. 查询某数据元素是否存在
 - 2. 检索某数据元素的各种属性
 - 3. 在查找表中插入一个数据元素
 - 4. 从查找表中删除一个数据元素
- 查找表的种类
 - 静态查找表 – 仅进行查找操作（1, 2）
 - 动态查找表 – 可进行插入、删除操作（1, 2, 3, 4）



线性查找表 - 顺序查找

- 从线性表的一端开始，依次把每个元素的关键字同给定值进行比较（也称线性查找）



- 假设每个元素的查找概率相等，则平均查找长度为：

$$ASL = \sum_{i=0}^{N-1} P_i C_i = \sum_{i=0}^{N-1} \frac{1}{N} (i + 1) = \frac{N + 1}{2}$$

- 对于长度为 N 的线性表，平均来说顺序查找需要进行 $\frac{N}{2}$ 次比较



线性查找表 - 顺序查找

- 没有利用每个元素的查找概率（如果不相等）
- 对于长度为 N 的无序表，查找失败需要 N 次比较
- 线性查找表一般不是按关键字有序排列的
- 如果线性查找表是从小到大有序排列的，那么当遇到关键字较给定值大的元素，即可确认查找失败
- 对于长度为 N 的有序表，查找失败需要 $N/2$ 次比较
- 顺序查找属于蛮力算法



线性查找表 - 折半查找

- 如果顺序表对于关键字是有序的，可采用折半查找（对分查找）
- 可用二叉树描述折半查找过程：折半查找树

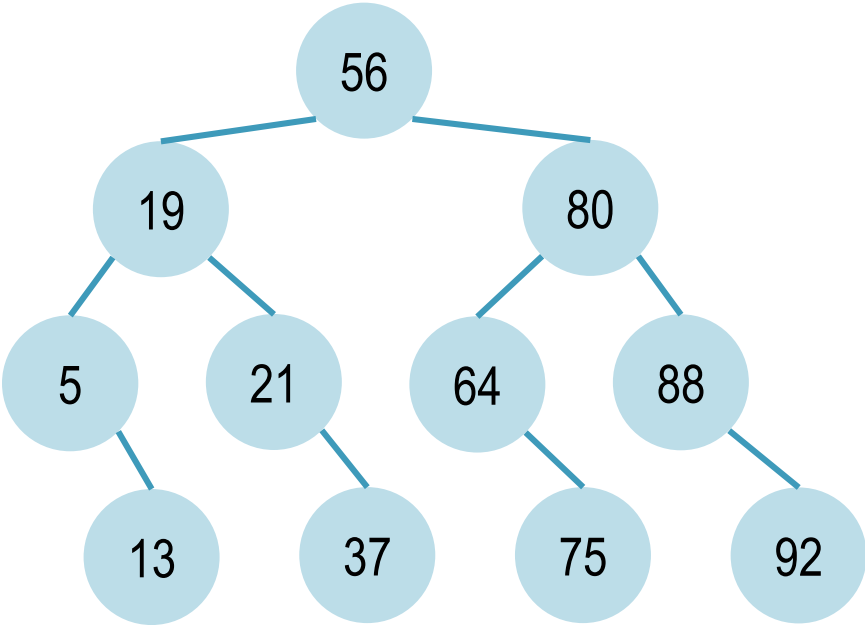
0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

- 起始位置: *left*
- 终止位置: *right*
- 中间位置: *mid*
- $l = 0, r = 10, m = 5, \dots$



线性查找表 - 折半查找

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

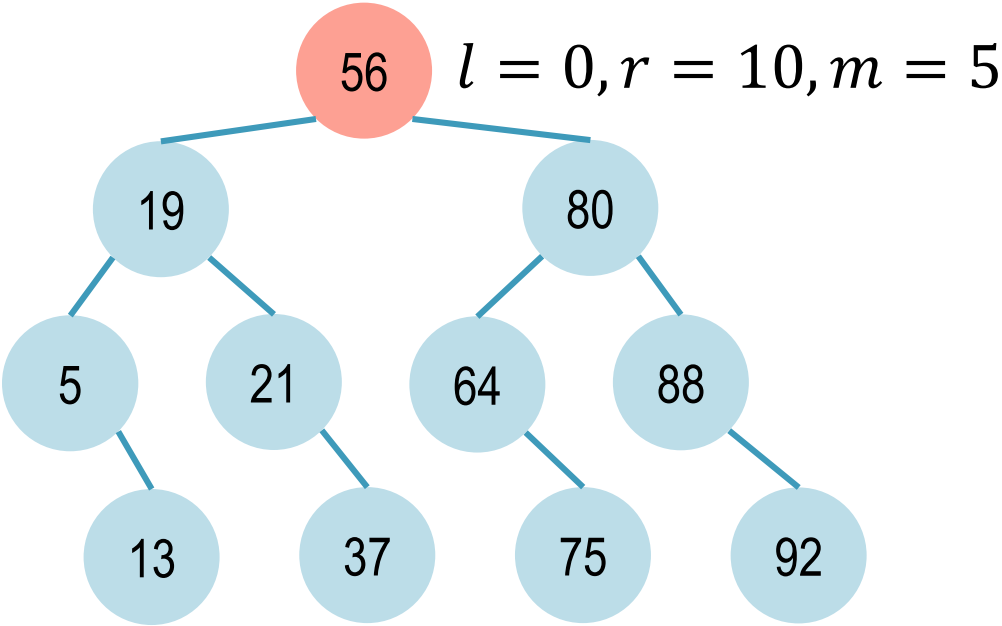




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

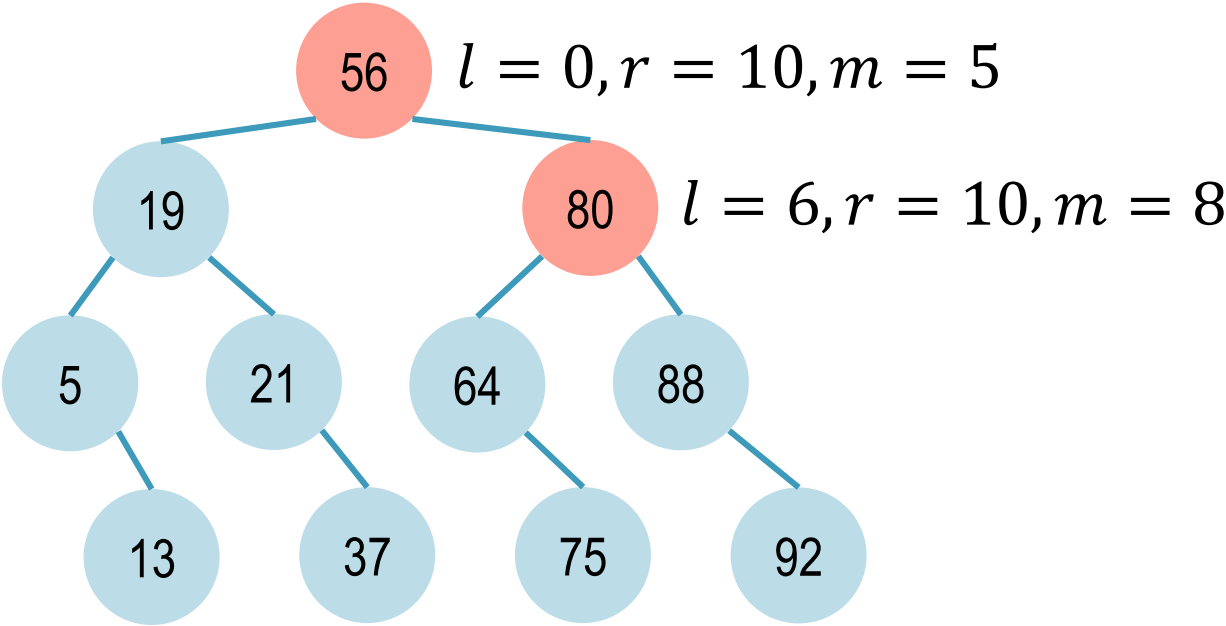




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

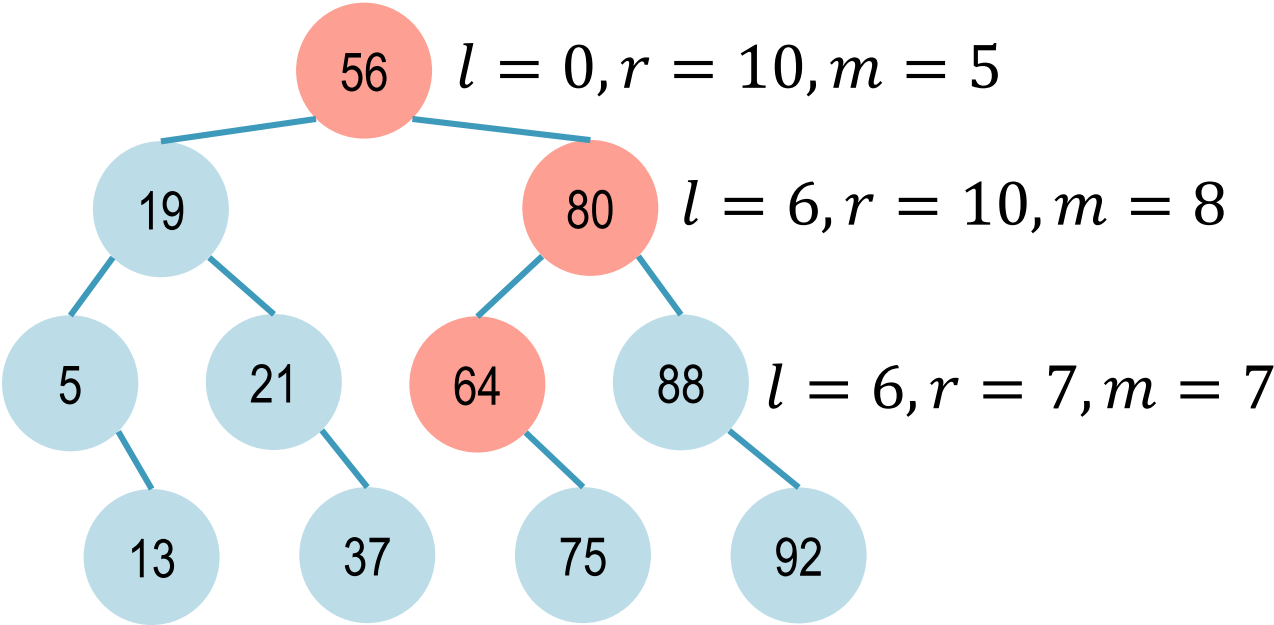




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

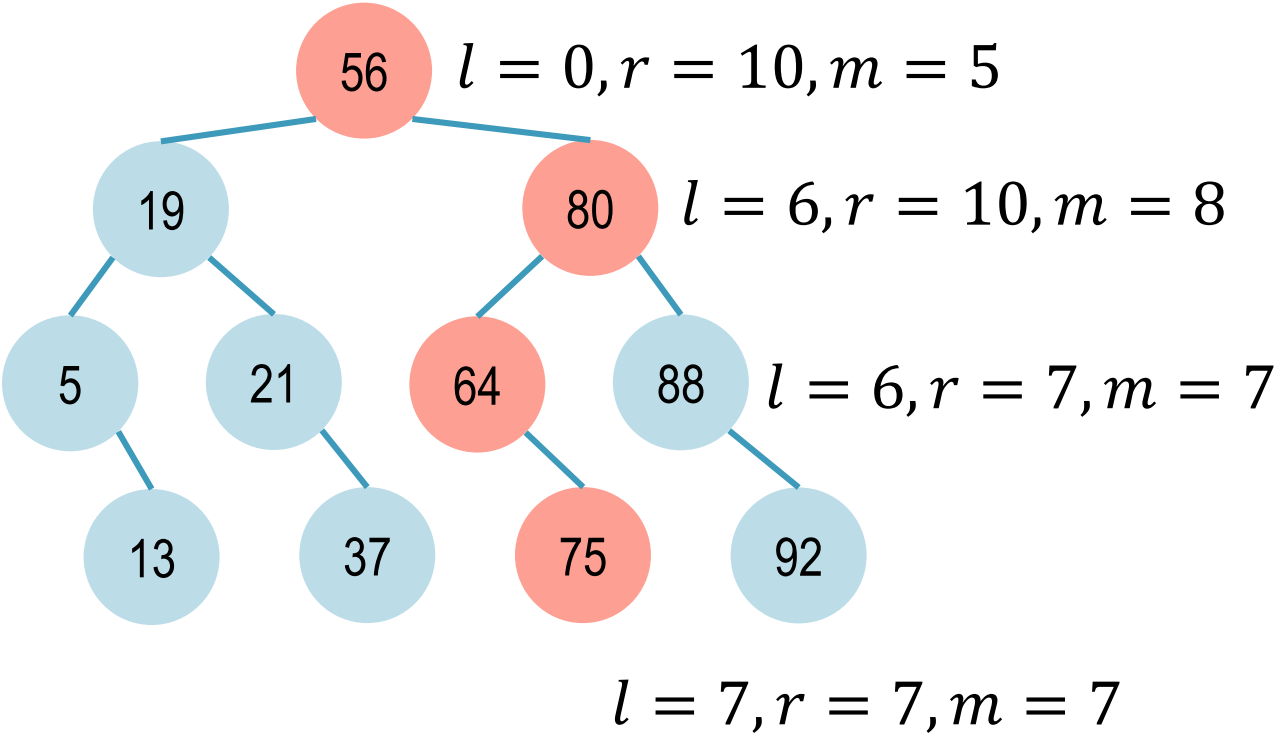




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92





线性查找表 - 折半查找

- 折半查找可以用一个二叉树结构来描述
- 二叉树的深度是折半查找需要的比较次数
- 对于一次查找，无论成功还是失败，折半查找需要的比较次数不会超过 $\lfloor \log_2 n \rfloor + 1$
- 折半查找的平均查找长度 $O(\log_2 n)$
- 折半查找属于减治算法



线性查找表比较

	查找	插入	删除
无序顺序表	$O(n)$	$O(1)$	$O(n)$
无序线性链表	$O(n)$	$O(1)$	$O(n)$
有序顺序表	$O(\log_2 n)$	$O(n)$	$O(n)$
有序线性链表	$O(n)$	$O(n)$	$O(n)$

- 有序顺序表的查找效率较高，但插入和删除的时间复杂度高
- 无序线性表的插入效率高
- 线性查找表更适合静态查找表

提纲

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法



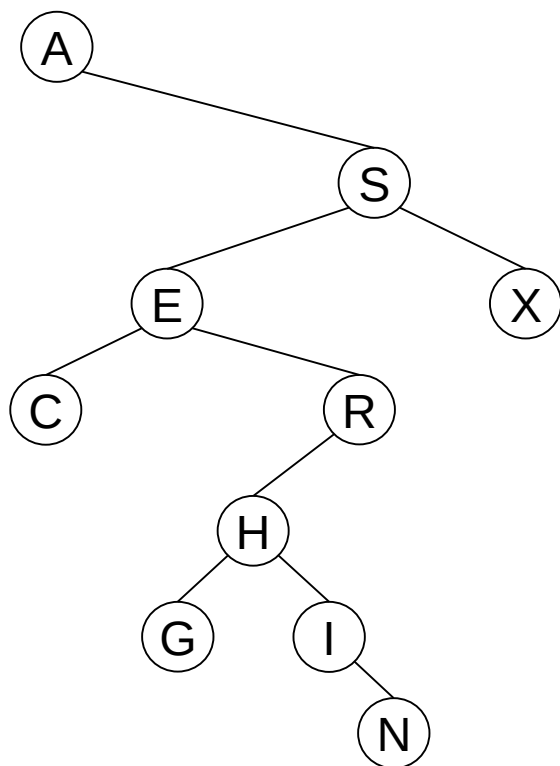


二叉搜索树

- 二叉搜索树是一种动态查找结构，效率可达 $O(\log_2 N)$
- 二叉搜索树 (Binary Search Tree, BST) 或者是一棵空树，或者是具有下列性质的二叉树
 - 每个结点有一个关键字 (key)
 - 任意结点关键字大于等于该结点左子树中所有结点的关键字
 - 任意结点关键字小于等于该结点右子树中所有结点的关键字
- 虽然二叉搜索树是非线性结构，但却隐含了一个线性有序序列
 - 中序遍历可以得到一个有序序列



二叉搜索树



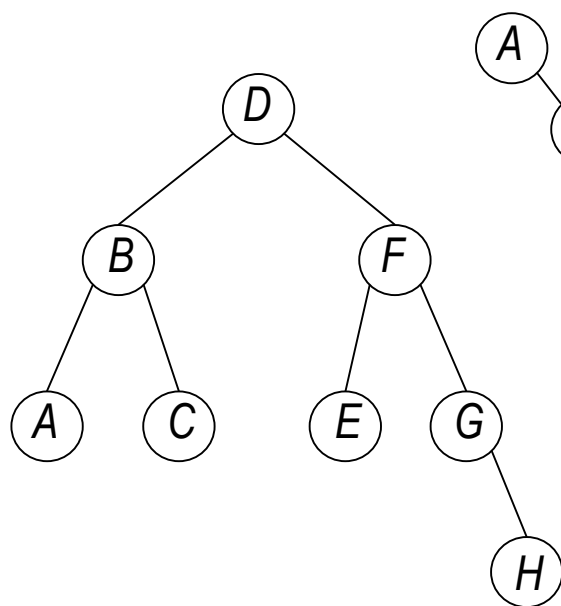
- 一棵二叉搜索树代表了一个经过排序的关键字序列，但不再是线性结构
- 插入与查找过程类似：查找失败后执行插入操作 → 高效的动态查找结构

中序遍历：A - C - E - G - H - I - N - R - S - X

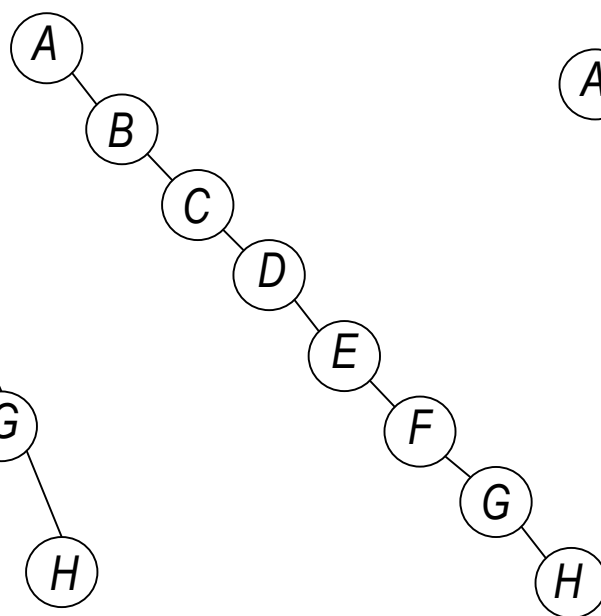


二叉搜索树

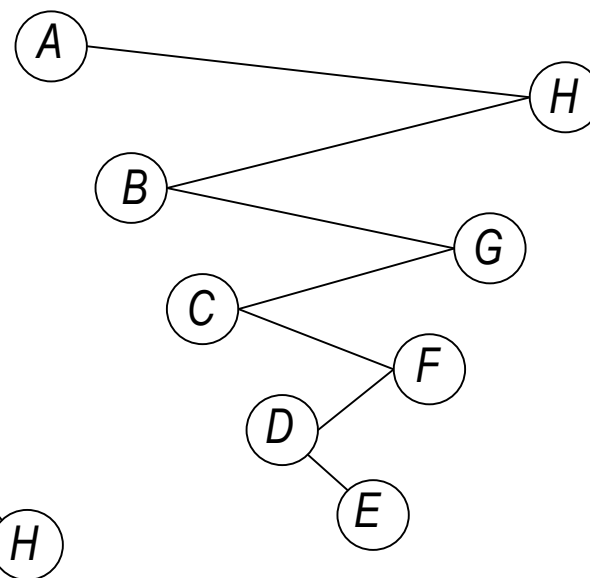
二叉搜索树查找算法的时间复杂度与树的形态有关



好的BST



坏的BST



坏的BST



二叉搜索树与线性查找表的比较

	平均情况		最坏情况	
	查找	插入	查找	插入
无序顺序表	$O(n)$	$O(1)$	$O(n)$	$O(1)$
无序线性链表	$O(n)$	$O(1)$	$O(n)$	$O(1)$
有序顺序表	$O(\log_2 n)$	$O(n)$	$O(\log_2 n)$	$O(n)$
有序线性链表	$O(n)$	$O(n)$	$O(n)$	$O(n)$
二叉搜索树	$O(\log_2 n)$	$O(\log_2 n)$	$O(n)$	$O(n)$

提纲

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法



索引查找

➤ 当线性表规模增加，具有线性复杂度

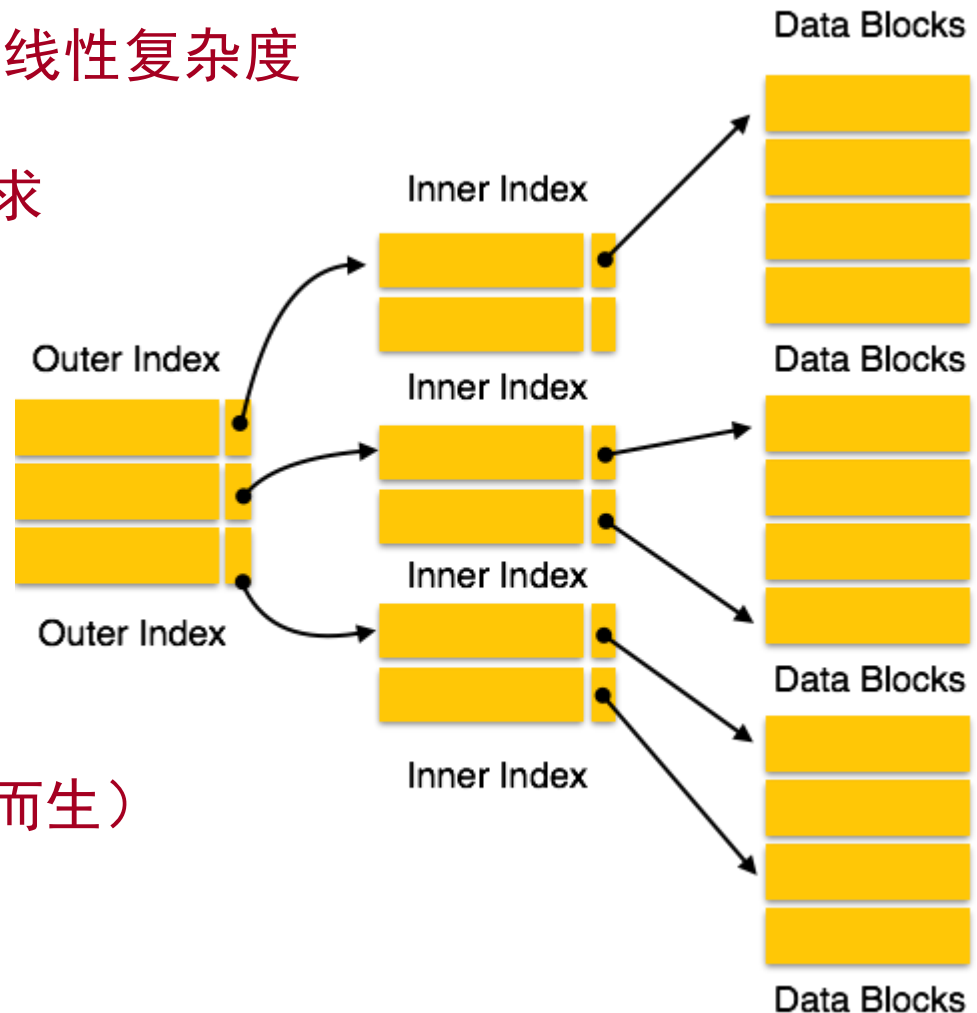
的线性查找仍难以满足需求

➤ 索引查找

- 图书检索目录
- 字典的检字表
- 书的章节目录

➤ 数据的索引结构（为查找而生）

- 实现索引查找的基础





索引结构

➤ 基本思想

- 将有 n 个结点的线性表划分为 m 个子表，分别存储各子表
- 另设一个有 m 个结点（索引项）的索引表，每个结点存子表第一个结点的地址及相关信息

➤ 线性表的划分原则

- 具有某种性质 P_i 的结点归并到子表 l_i 中
- m 个子表的结点合并起来，正好构成原线性表的全部结点

➤ 索引函数 g

- 能对表中每个关键字 k 和其在索引表中的序号 i 之间建立对应关系
- $i = g(k)$
- 索引查找关键是确定索引函数



索引结构举例

将标称值为200Ω，实际值为190~210 Ω的电阻，按实际测量值分为7组

	l_1	l_2	l_3	l_4	l_5	l_6	l_7
范围	190-192	193-195	196-198	199-201	202-204	205-207	208-210

索引函数： $i = \text{INT}[(R - 190)/3] + 1$

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
阻值	193	197	191	201	207	199	204	194	208	202

分类结果

l_1	l_2	l_3	l_4	l_5	l_6	l_7
R_3	$R_1 R_8$	R_2	$R_4 R_6$	$R_7 R_{10}$	R_5	R_9



索引存储方式

- 索引结构是静态查找结构，数据组织上直接简单，对插入删除等动态操作的支持没有很高要求
- 索引表和子表均可用顺序表或者链表进行存储
 - 顺序索引-顺序存储
 - 顺序索引-链接存储
 - 链接索引-顺序存储
 - 链接索引-链接存储
- 索引表规模不大，元素变动少
 - 一般采用顺序表来存储索引表

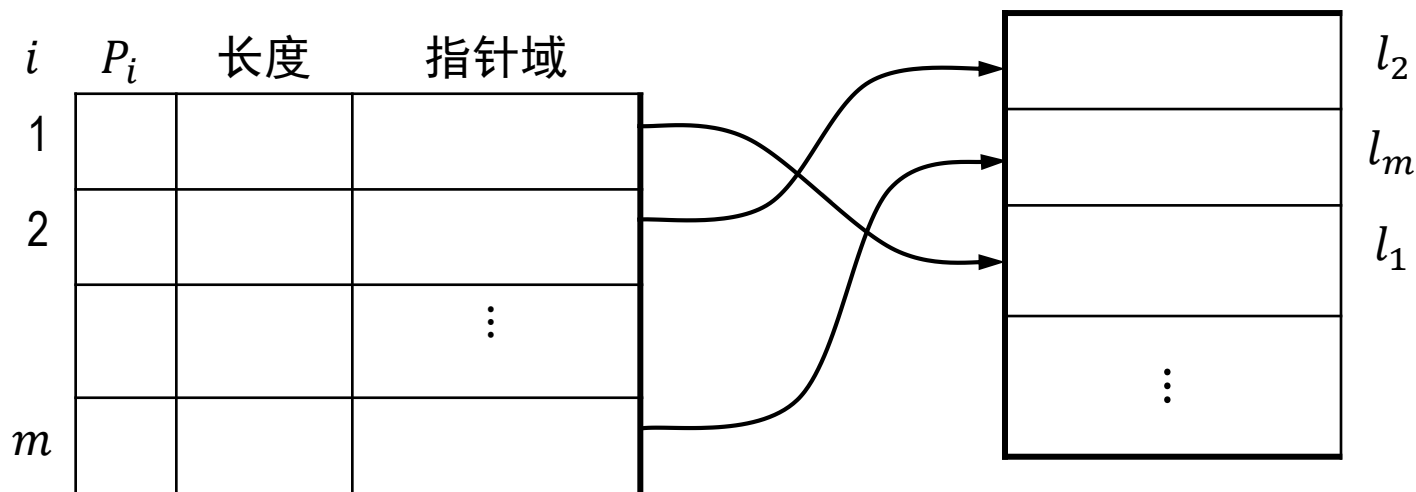


顺序索引 - 顺序存储

- 索引表和子表均为顺序表
- 子表顺序存放，子表间无存储顺序要求

索引结点结构：

信息域		指针域
P_i	长度	子表第一个结点地址





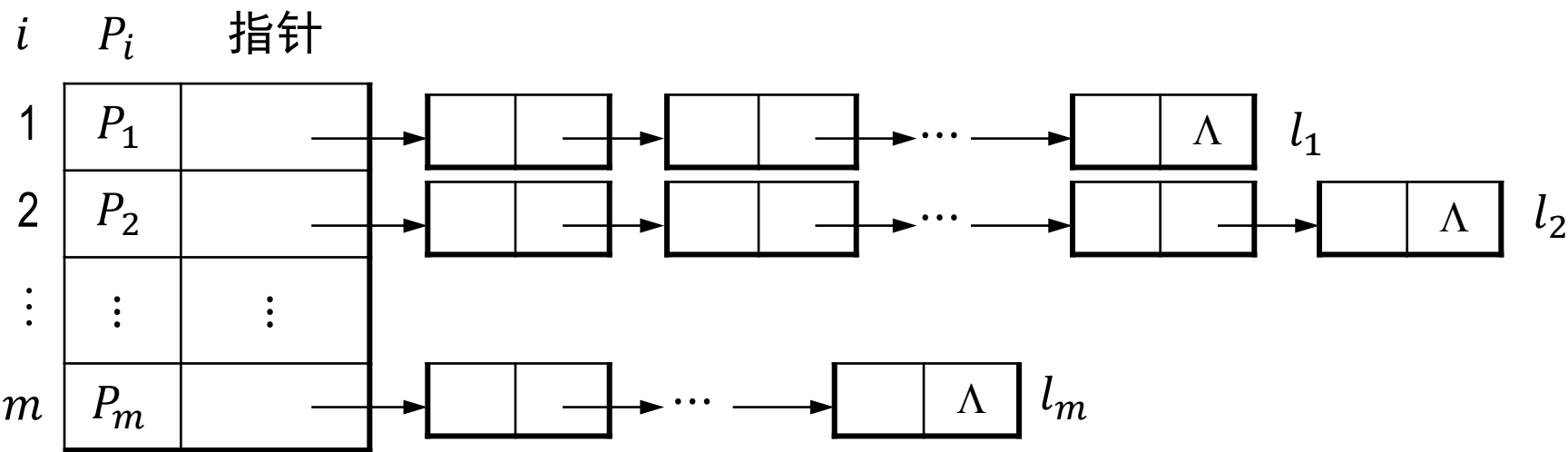
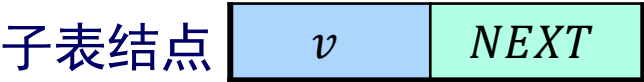
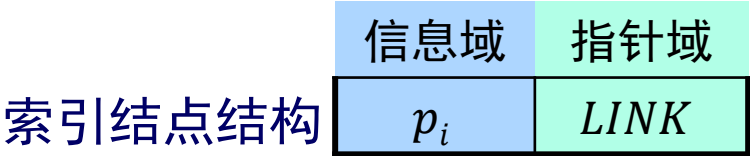
顺序索引 - 顺序存储 - 查找步骤

- 计算 $i = g(k)$
- 读出索引表中第 i 个索引项的指针项、子表长度值
- 若子表 i 长度为0，则查找失败；否则，在子表 i 中顺序查找，若所查元素不在表中，则由子表长度控制查找结束，查找失败
- 两级查找过程
 - 先查（算）索引项，再查子表
 - 通过索引使顺序查找规模缩小
- 索引查找比全表查找效率高



顺序索引 - 链接存储

索引表为顺序表，子表为链表





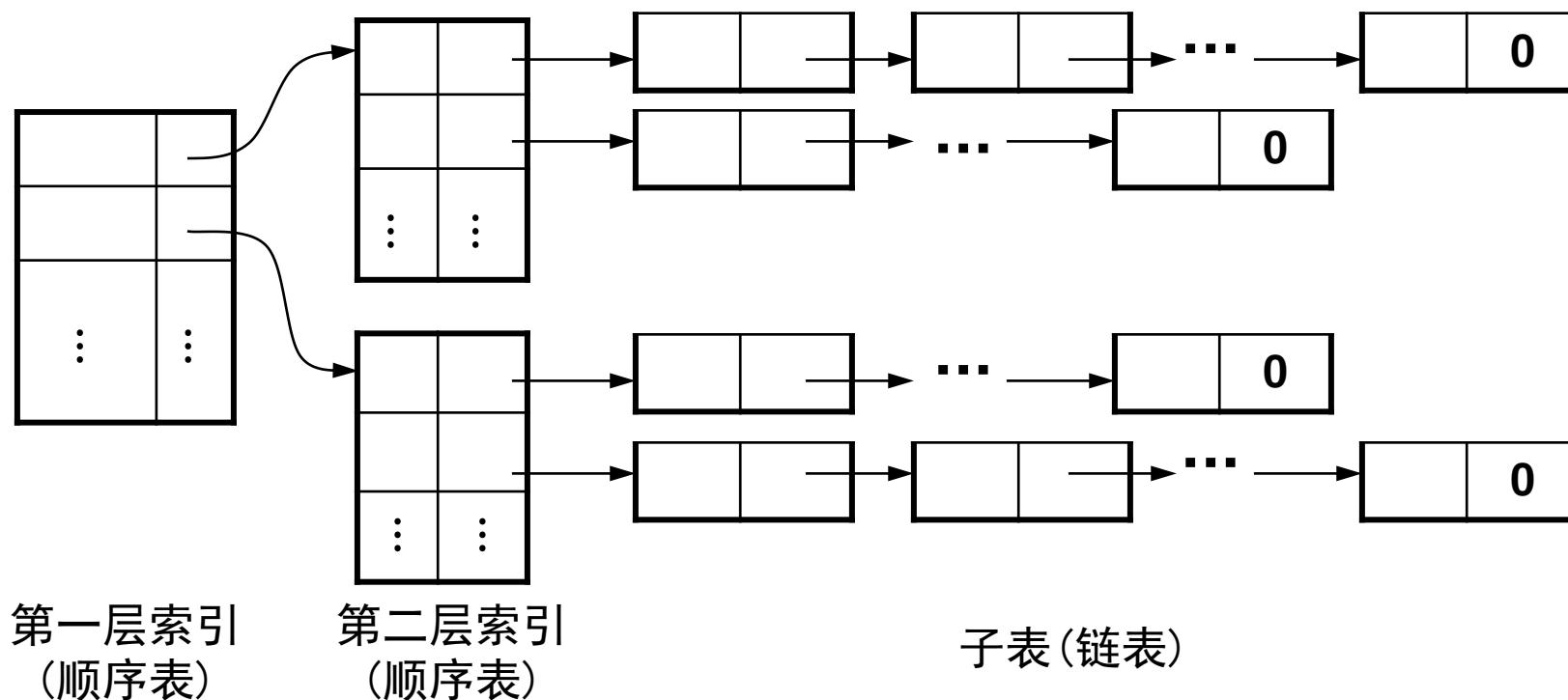
顺序索引 - 链接存储 - 查找步骤

- 计算 $i = g(k)$
- 读出索引表中第 i 个索引项的指针项
- 在第 i 个子表中**顺链查找**
 - 若找到关键字为 k 的元素，查找成功
 - 若搜索完全链未找到关键字为 k 的元素，查找失败
- 索引表结构简单，便于查找
- 子表结点插入删除不需要移动表中元素



多重索引

- 当索引表很大时，可对索引表再做索引，进一步提高查找效率
- 举例：顺序索引-顺序索引-链接存储



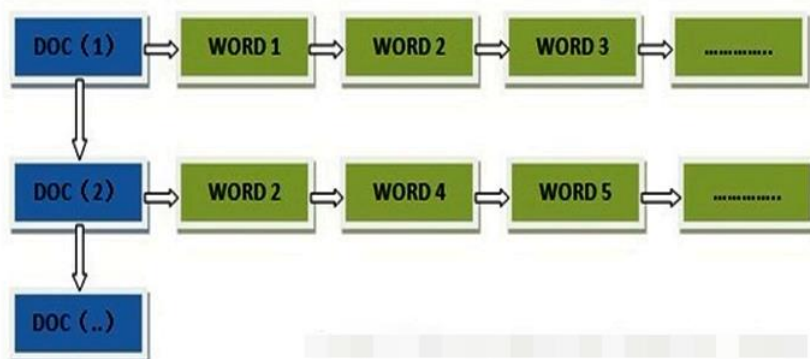


索引文件结构

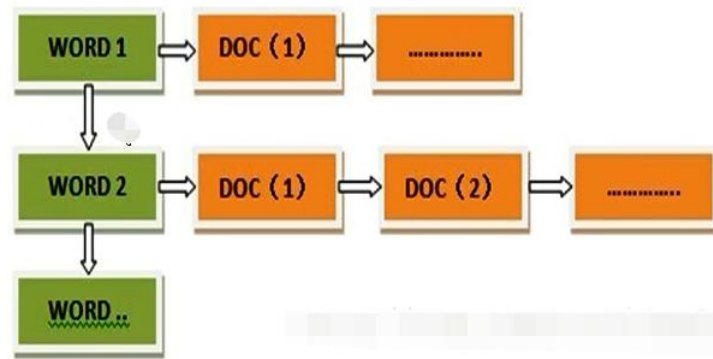
- 文件：具有相同性质的元素的集合
 - 保存在外存储器
 - 文件规模较大，无法一次全部调入内存
 - 直接顺序查找会导致反复读取外存，查找效率低
- 索引文件：具有索引存储结构的文件
- 索引文件结构
 - 主文件：原有数据文件
 - 索引表：指示文件中元素与存放物理地址关系的表
 - 索引表中每一项称为索引项
 - 各索引项按各元素的关键字有序排列

索引文件结构

- 单关键字索引文件结构：用元素中某一项作为关键字建立索引文件
- 多关键字索引文件结构：用多个关键字建立索引文件
- 倒排文件：通过关键字快速获取文件列表（给定几个关键字，找出包含关键词的文件）



正向索引：从文件到关键字



倒排索引：从关键字到文件
(搜索引擎广泛使用)

提纲

➤ 图的最短路径问题

➤ 查找

- 线性查找表
- 二叉搜索树
- 静态索引查找
- 散列方法





散列方法的基本思想

- 查找技术一般是基于待查关键字与数据元素关键字的比较，所有关键字比较方法计算复杂度： $O(\log_2 n) \sim O(n)$
- 索引结构计算 $i = g(k)$ ，使查找规模缩小，比全表查找效率高

找一个好的函数 g ，直接根据关键字把地址算出来？

- 本班级42名同学，以学号为关键字，可以用一个长度为42的数组来存储
- 也可以？

2021012852 ~ 2023080087: $2023080087 - 2021012852 = 2067235$

建长度为2067235的数组A[], 学号i的学生信息存储在A[i - 2021012852]



散列方法的基本思想

- 查找技术一般是基于待查关键字与数据元素关键字的比较，所有关键字比较方法计算复杂度： $O(\log_2 n) \sim O(n)$
- 索引结构计算 $i = g(k)$ ，使查找规模缩小，比全表查找效率高

找一个好的函数 g ，直接根据关键字把地址算出来？

- 直接采用关键字“作为”（计算）存储“序号”
- 关键：关键字和存储位置如何对应？
- 散列函数是将关键字集合映射到某个地址集合上的映射



一个例子：乘法散列函数

浮点型关键字	索引	浮点型关键字	索引
0.513870656	50	0.277230144	27
0.175725579	17	0.368053228	36
0.308633685	30	0.983458996	95
0.534531713	52	0.535386205	52
0.947630227	92	0.765678883	74
0.171727657	17	0.646473587	63
0.702230930	68	0.767143786	74
0.226416683	22	0.780236185	76
0.494766086	48	0.822962105	80
0.124698631	12	0.151921138	15
0.838953850	81	0.625476837	61
0.389629811	38	0.314676344	31

➤ $Hash(k) = \text{int}(Mk),$
 $k \in (0,1)$

➤ $Hash(k) = \text{int}[M \frac{(k-s)}{t-s}],$
 $k \in (s, t)$

➤ 只有当关键字在一定范围内平均分布时，这种方法才是高效的



散列方法

- 散列技术核心是数据元素存放位置和关键字之间的关系
 - 散列函数
 - 冲突处理：希望不同关键字映射到不同地址，实际经常出现映射到同一个地址的情况，即“冲突”，散列技术的关键是冲突调节
- 散列表(Hash表)将一组关键字映射到一个有限的、地址连续的区间上，并以关键字在地址集中的“象”作为相应记录在表中的存储位置
- Hash表规模远大于元素的数目，冲突容易避免，查找操作时间复杂度低，但空间复杂度高；反之，如果Hash表的规模和元素数目相当，空间复杂度低，但冲突很难避免，需消耗时间做处理冲突
- 设计关键：存储量与查找时间的合理平衡



散列方法 - 散列函数

散列函数的构造原则

- 简单，能在较短时间内计算出结果
- 定义域必须包含全部关键字
- 若散列表允许有 m 个地址，其值域必须在 $0 \sim m - 1$ 之间
- 理想的散列函数应近似随机
- 对每一个输入，输出在值域上等概率（因为地址是均匀的），利于减少冲突发生



模散列函数

- 关键字 k 是整数

$$\text{Hash}(k) = k \bmod M$$

- M 不大于表长
- M 为素数，或至少没有小于20的因子，有利于减少冲突

- 可以把乘法与取模两种算法结合起来

- 关键字 k 是整数

$$\text{Hash}(k) = [ka] \bmod M$$

- $a \in (0,1)$ ，常取0.618033

- 还有将乘法（Multiply）、加法（Add）和除法（Divide）结合起来的方法

$$\text{Hash}(k) = [ka + b] \bmod M$$

- 称为MAD法



模散列函数

整型 关键字	k%97	k%100	$(\text{int})(k*a)\%100$	整型 关键字	k%97	k%100	$(\text{int})(k*a)\%100$
16838	57	38	6	9084	63	84	14
5758	35	58	58	12060	32	60	53
10113	25	13	50	32225	21	25	16
17515	55	15	24	17543	83	43	42
31051	11	51	90	25089	63	89	5
5627	1	27	77	21183	37	83	91
23010	21	10	20	25137	14	37	35
7419	47	19	85	25566	55	66	0
16212	13	12	19	4978	31	78	76
4086	12	86	25	20495	28	95	66
2749	33	49	98	10311	29	11	72
12767	60	67	90	11367	18	67	25



数字分析法

- 设有 n 个 d 位数,每一位可能有 r 种不同的符号。这 r 种不同的符号在各位上出现的频率不一定相同。可根据散列表的大小,选取其中各种符号分布均匀的若干位作为散列地址
- 举例: 学生学号后三位
- 数字分析法仅适用于事先明确知道表中所有关键字每一位数值的分布情况,它完全依赖于关键字集合。如果换一个关键字集合,选择哪几位要重新决定



平方取中法

- 先计算构成关键字的标识符的内码（内码是指操作系统内部的字符编码，与ASCII码兼容）的平方，然后按照散列表的大小取中间的若干位作为散列地址
- 平方取中法的性能较好，因为 k 的平方值中间几位数与 k 的每一位都有关
- 有研究认为平方取中法的输出最接近于“随机化”
- 平方取中法的计算比较复杂

平方取中法，亦称冯·诺伊曼取中法，最早由冯·诺伊曼(John von Neumann, 1903-1957)提出的一种产生均匀伪随机数的方法



平方取中法

标识符	内码（八进制）	内码的平方	散列地址
A	01	1	1
A1	0134	20420	42
A9	0144	23420	342
B	02	4	4
DMAX	04150130	21526443617100	443
DMAX1	0415013034	5264473522151420	352
AMAX	01150130	135423617100	236
AMAX1	0115013034	3454246522151420	652

散列方法的冲突处理

➤ 冲突：两个关键字散列到同一个地址

- 冲突不可避免

➤ 冲突处理方法

- 开散列法：链地址法

- 闭散列法：开放定址法





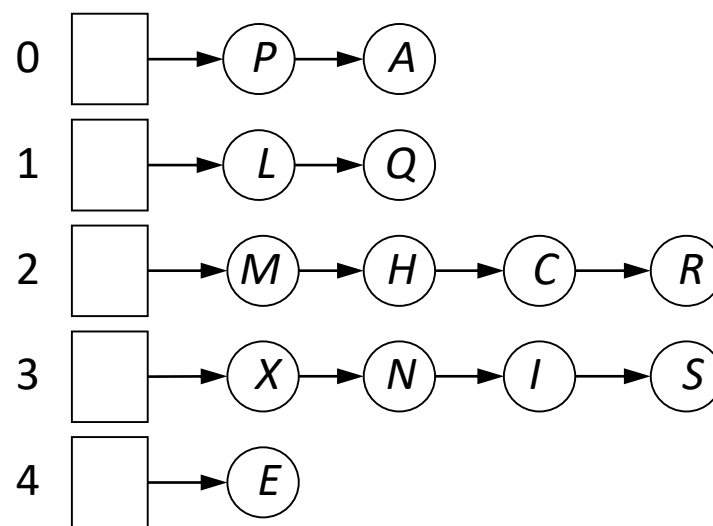
开散列法 - 链地址法

- 对每个散列地址建立一个链表，将散列到同一个地址的不同关键字存入相应的链表中

A	S	E	R	C	H	I	N	Q	X	M	P	L
65	83	69	82	67	72	73	78	81	88	77	80	76
0	3	4	2	2	2	3	3	1	3	2	0	1

$$\text{Hash}(c) = \text{ASCII}(c) \bmod 5$$

- 元素数目 N ，模散列法 M 取余
- M 个地址链， N 个关键字
- 装载因子： $\alpha = \frac{N}{M}$
- 链地址法中 $\alpha > 1$ ，表示链表平均长度



$$N = 13, M = 5$$



开散列法 - 链地址法

- 比较次数减少为顺序查找的 M 分之一
- 链地址法空间利用率高，只需 M 个链表头的额外空间
- 当空间不是关键性资源时，不妨选择足够大的 M 以使搜索时间为常数
- 当空间至关重要时，可以在能够承受范围中选择 M
- 取 M 尽量小以避免空地址链浪费大量连续的内存空间
- 一般选择 M 大约为数据元素数目的 $1/5$ 到 $1/10$ ，这样，链表长度平均值 $5\sim 10$



闭散列法 - 开放定址法

- 开放定址法：对表长 M 大于元素数目 N 的情况，依靠存储空间冗余来解决冲突问题，使得所有元素存储在Hash表已经分配的空间
- 检查给定位置上是否存在一个与待搜索关键字不同的元素，称为探测（开放定址法的基本操作）；如果已有别的元素，寻找别的位置
- 线性探测：检查表中的下一个位置，最简单的开放定址法



闭散列法 - 开放定址法

散列建立过程

- 根据散列函数，计算位置
- 如果表位置上为空，插入元素
- 如果表位置上有元素，则继续探测表的后续位置（如果搜索到表尾要回到表头继续搜索）
- 找到空位置，插入元素

[illegible]



闭散列法 - 开放定址法

- 查找方法与建Hash表方法一致
- 对于给定值 k , 计算散列地址 $i = Hash(k)$
 - 若 $a[i] = NULL$ 则查找不成功
 - 若 $a[i].key = k$ 则查找成功
 - 否则 求下一地址 H_i , 直至
 - $a[H_i] = NULL$ (查找不成功)
 - $a[H_i].key = k$ (查找成功)



闭散列法 - 开放定址法 - 性能分析

- N 为元素数目, M 为表长, 定义负载因子 $\alpha = N/M$
 - 对链地址法来说, α 是每个地址链的平均长度, 大于1
 - 对开放定址法来说, α 表示表中位置被占据的概率, 它一定是小于1的。
- 开放定址法的性能依赖于负载因子
 - α 很小时, 称为稀疏表, 我们期望大多数搜索只需几次探测就能找到表的空位
 - α 接近1时, 散列表接近满, 一次搜索将需要相当多次的探测
 - 为了避免过长的搜索时间, 通常控制表在半满状态以下



Hash表的删除

- 链式地址法：查找到后从链表中删除
- 开放定址法：直接删除会出问题，因为元素有链接元素的功能，可将被删除关键字加标志留在散列表，或满足一定条件时重新散列

$$Hash(k) = ASCII(k) \text{ Mod } 13$$

<i>H</i>	<i>D</i>	<i>J</i>	<i>W</i>	<i>I</i>	<i>R</i>	<i>L</i>	<i>U</i>	<i>X</i>	<i>M</i>	<i>A</i>	<i>V</i>	
72	68	74	87	73	82	76	85	88	77	65	86	
7	3	9	9	8	4	11	7	10	12	0	8	
<i>X</i>	<i>M</i>	<i>A</i>	<i>D</i>	<i>R</i>	<i>V</i>		<i>H</i>	<i>I</i>	<i>J</i>	<i>W</i>	<i>L</i>	<i>U</i>
<i>X</i>	<i>M</i>	<i>A</i>	<i>D</i>	<i>R</i>	<i>V</i>		<i>H</i>	<i>I</i>	<i>J</i>	<i>W</i>		<i>U</i>
0	1	2	3	4	5	6	7	8	9	10	11	12

删除*L*前

删除*L*后
无法找到*U*等



散列方法 - 性能

- 决定散列表查找性能的因素：
 - 选用的散列函数
 - 选用的处理冲突方法
 - 散列表饱和的程度，装载因子 $\alpha = N/M$ 值的大小
- 在保证散列表足够稀疏的情况下，线性探测散列具有最快的查找性能；当散列表趋近于满时，性能下降较快
- 链地址法容易实现，不会随着表中元素的增加而出现性能迅速下降的情况，但其平均情况下的时间性能不如开放定址法



本讲小结

- 图的最短路径问题
- 查找（查找需求→数据结构）
 - 线性查找表： $O(\log_2 n)$ vs. $O(n)$
 - 二叉搜索树： 动态查找结构
 - 静态索引查找： 索引表（索引函数） + 子表
 - 散列方法： 散列函数 + 冲突处理

加油同学们!

